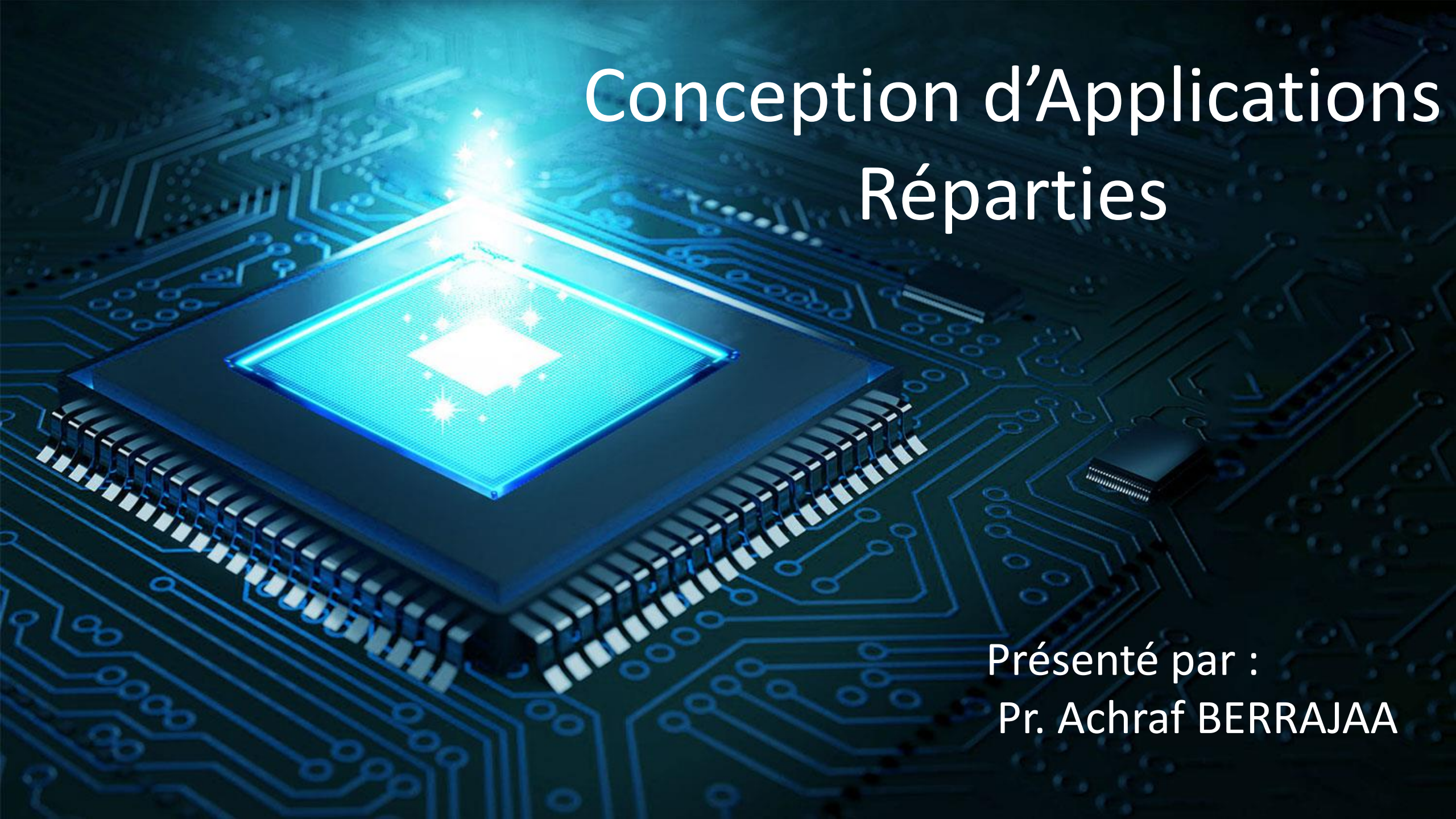




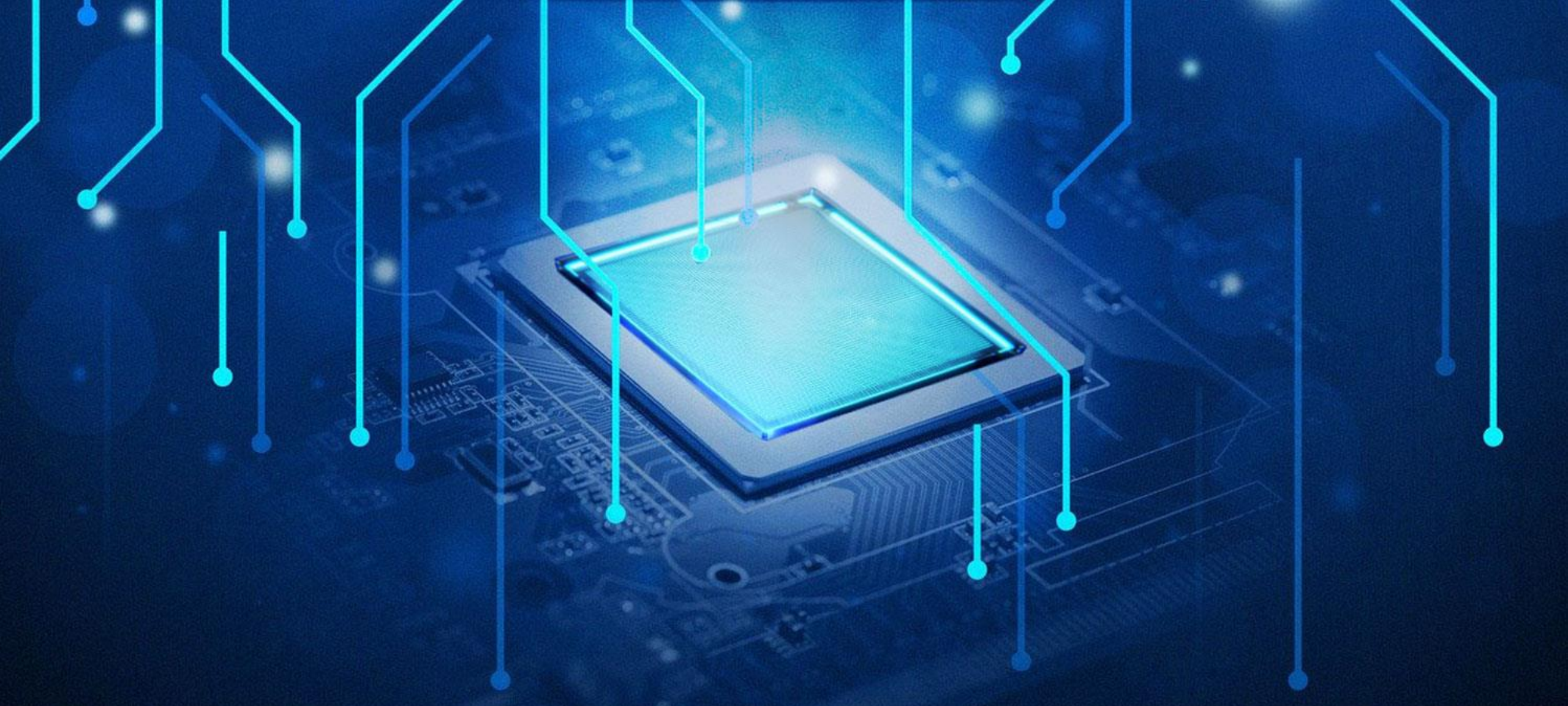
Conception d'Applications Réparties

Présenté par :
Pr. Achraf BERRAJAA

Pourquoi ce cours ?

C'est quoi la programmation répartie ?

La programmation répartie (ou programmation distribuée) vise à concevoir et à développer des systèmes informatiques dans lesquels **plusieurs ordinateurs ou processus interagissent pour atteindre un objectif commun.**



La programmation répartie

Plan du cours

01

Introductions aux applications réparties

02

Programmation avec les Sockets (TCP/UDP)

- Sockets en C
- Sockets en Java

03

Middleware et Systèmes Répartis:

- RPC
- Java RMI

04

MPI (Message Passing Interface)

05

La programmation des processeurs graphique : GPU (Graphical Processing Unit)

Introduction aux applications réparties



01

Ch. I. Introduction aux applications réparties

1. Systèmes répartis :

- **Système centralisé** : tout est localisé sur la même machine et accessible par le programme
 - L'exécution s'effectue sur une seule machine
 - Accès local aux ressources (données, code, périphériques, mémoire ...)
- **Système réparti** : Un système réparti est composé de plusieurs nœuds (ordinateurs, serveurs, capteurs, etc.), **chacun ayant une capacité de calcul indépendante**.
 - Les nœuds **échangent** des informations via **des protocoles de communication réseau** (par exemple, TCP/IP).
 - Contrairement aux systèmes centralisés, les processus dans un système réparti ne partagent pas directement la mémoire. **Les échanges se font par messages** ou via des bases de données réparties.

Ch. I. Introduction aux applications réparties

Définition 1 : Le calcul parallèle consiste à faire coopérer plusieurs unités de calcul (processeurs, cœurs, ...) pour résoudre le même problème (application). Il consiste à partitionner un gros problème en petits problèmes indépendants.

Définition 2 : Le calcul parallèle est l'ensemble des techniques logicielles et matérielles permettant l'exécution d'un programme en utilisant plusieurs unités de calcul plutôt qu'une seule.

Définition 3 : Une machine (un ordinateur) parallèle est une machine composée de plusieurs processeurs qui peuvent communiquer entre eux.

Architecture mono processeur: Un seul processeur qui exécute séquentiellement les instructions d'un programme.



Les instructions sont exécutées l'une après l'autre.

Architecture parallèle: Plusieurs processeurs exécutent en parallèle le même programme.



Des séquences d'instructions indépendantes sont exécutées simultanément.

Ch. I. Introduction aux applications réparties

Pourquoi utiliser un système réparti ?

Les systèmes répartis sont populaires pour plusieurs raisons:

- **Accès distant:** un même service peut être utilisé par plusieurs acteurs, situés à des endroits différents
- **Redondance:** des systèmes redondants permettent de pallier une faute matérielle, ou de choisir le service équivalent avec le temps de réponse le plus court
- **Performance:** la mise en commun de plusieurs unités de calcul permet d'effectuer des calculs parallélisables en des temps plus courts
- **Partage des données:** les systèmes ont besoins de communiquer entre eux des données
- **Capacité d'évolution :** un système réparti peut être évoluer facilement en ajoutant d'autres service ou d'autres composants

Ch. I. Introduction aux applications réparties

Pourquoi utiliser un système réparti ?

Les systèmes répartis sont populaires pour plusieurs raisons:

- Les systèmes répartis sont équipés d'une couche logiciel dédié à la coordination des activités du système ainsi qu'au partage des ressources.
 - Pour remédier au problème de l'hétérogénéité des ressources matériels et logiciels (ordinateur, systèmes d'exploitation, langage de programmation).
 - Pour cacher la complexité du matériel et des communications.
 - Pour fournir des services communs de plus haut niveau d'abstraction.
- Du point de vue de l'utilisateur, le système apparaît comme une unique entité.

Ch. I. Introduction aux applications réparties

2. Applications réparties :

Une application répartie est une application composée de plusieurs entités s'exécutant indépendamment et en parallèle sur un ensemble d'ordinateurs connectés en réseau pour répondre à un problème spécifique en utilisant les services généraux fournis par le système.

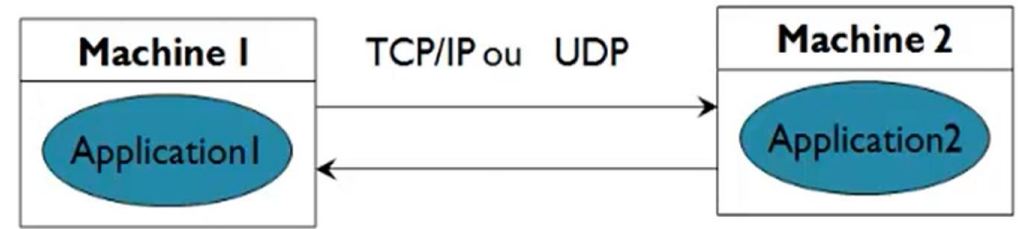
Les applications réparties

- **Ne se résument pas à l'échange de données en réseau.**
Exemple recherche sur le web: un navigateur Internet
- **Mais mettent en œuvre réellement une coopération des programmes entre eux.**
Échange à l'intérieur des réseaux d'entreprise: outils de production industrielle

Ch. I. Introduction aux applications réparties

Une application découpée en plusieurs unités

- Chaque unité peut être placée sur **une machine différente (ou sur la même machine)**
- Chaque unité peut s'exécuter sur **un système différent**
- Chaque unité peut être programmée dans **un langage différent**



Technologies d'accès :

- Sockets ou DataGram
- RMI (JAVA)
- CORBA (Multi Languages)
- EJB (J2EE)
- Web Services (HTTP+XML)

Ch. I. Introduction aux applications réparties

3. Avantages et Inconvénients de la répartition :

Avantages :

- **Gain en rapidité:** augmenter la puissance des calculs ce qui entraîne la réduction des temps d'exécution. Théoriquement, avec p processeurs on calcule plus vite qu'avec un seul processeur. Dans le cas idéal, avec p processeurs, le temps d'exécution est p fois plus rapide qu'avec un seul processeur.
- **Gain en taille mémoire:** Théoriquement, avec p processeurs, on dispose de p fois plus de mémoire, ce qui permet de résoudre des problèmes de taille plus grande.
- **Gain en tolérance aux fautes :** Système plus robuste.
 - Décentraliser les responsabilités: individualisation des défaillances.
 - Duplication (redondance) d'une application afin de diminuer le taux de pannes et augmenter la fiabilité : deux serveurs de fichiers dupliqués, avec sauvegarde.
- **Besoin de Performance**

Ch. I. Introduction aux applications réparties

3. Avantages et Inconvénients de la répartition :

Inconvénients :

- **La gestion du système est totalement décentralisée et distribuée**
 - Une mise en œuvre plus délicate
 - Gestion des erreurs
 - Suivi des exécutions
 - Pas de vision globale instantanée
- **Délais des transmissions:** si le débit d'information est très important
 - Goulot potentiel d'étranglement
- Les applications réparties imposent un mode de développement spécifique, et alourdissent le test et le déploiement du logiciel.

Ch. I. Introduction aux applications réparties

3. Avantages et Inconvénients de la répartition :

Inconvénients :

- **Problèmes réseaux:**
Si problème au niveau du réseau, le système peut tomber en panne.
- **Problème serveur:**
En générale un serveur est central au fonctionnement du système. Si le serveur plante, le système ne fonctionne pas.
- **Administration plus lourde:**
Installation
Configuration
Surveillance
- **Sécurité :** Accès aux données confidentielles. Authentification. Utilisation des ressources (périphériques, logiciels licenciés). Malveillance.

Ch. I. Introduction aux applications réparties

4. Exemples d'applications réparties :

Serveur de fichiers: Accès aux fichiers de l'utilisateur quelque soit la machine utilisée

- Physiquement, les fichiers se trouvent uniquement sur le serveur
- Virtuellement, l'accès à ces fichiers à partir de n'importe quelle machine cliente
- Intérêts
 - Accès aux fichiers à partir de n'importe quelle machine
 - Système de sauvegarde associé à ce serveur
 - Transparent pour l'utilisateur
- Inconvénients
 - Si le réseau ou le serveur plante on ne peut plus accéder aux fichiers.

Ch. I. Introduction aux applications réparties

4. Exemples d'applications réparties :

- **Web:** Un serveur web auquel se connecte différents navigateurs web. Accès transparent à distance à l'information. Les informations s'affichent dans son navigateur quelque soit la façon dont le serveur les génère.
 - Accès simple
 - Serveur renvoie une page HTML statique qu'il stocke localement
 - Traitement plus complexe
 - Serveur interroge une base de données pour générer dynamiquement le contenu de la page
- **Commerce électronique:** Transactions commerciales entre clients et fournisseurs.
- **Systèmes de réservation** de places (hôtels, avions, trains, concerts)
- **SGBD** avec accès multiple par des guichets ou des GAB (banque, PTT, assurances, mutuelles)

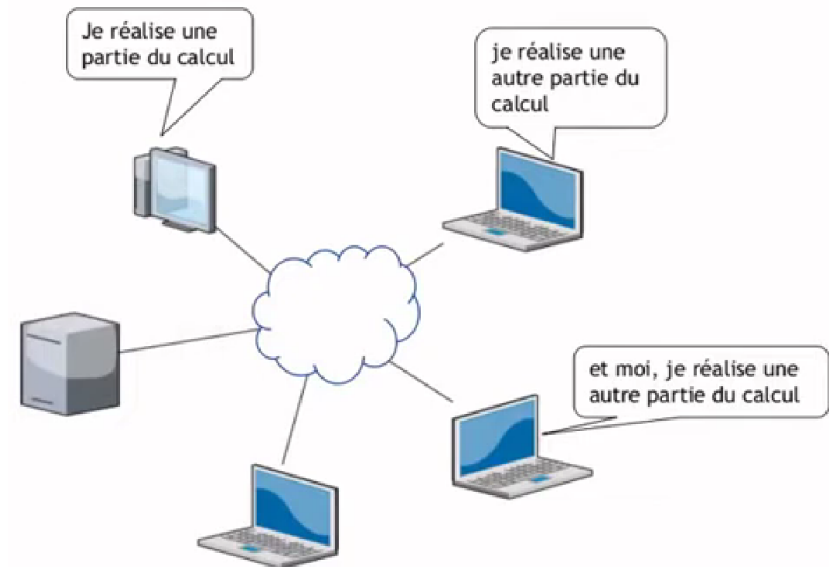
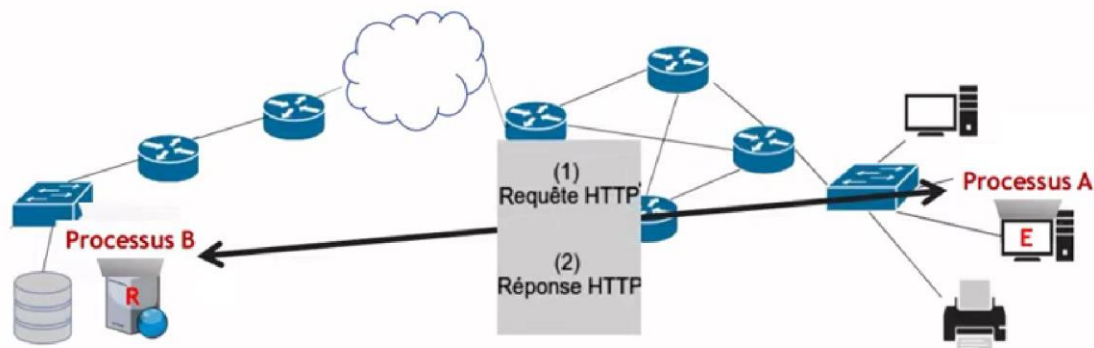
Ch. II. Programmation avec les Sockets

4. Exemples d'applications réparties :

La communication

Echange des données.

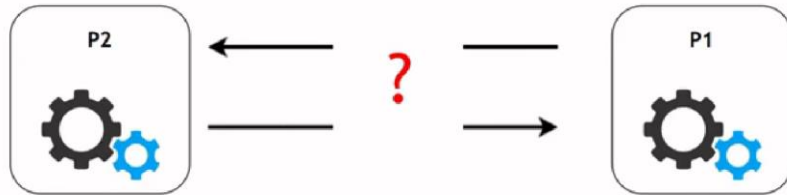
Calcul repart



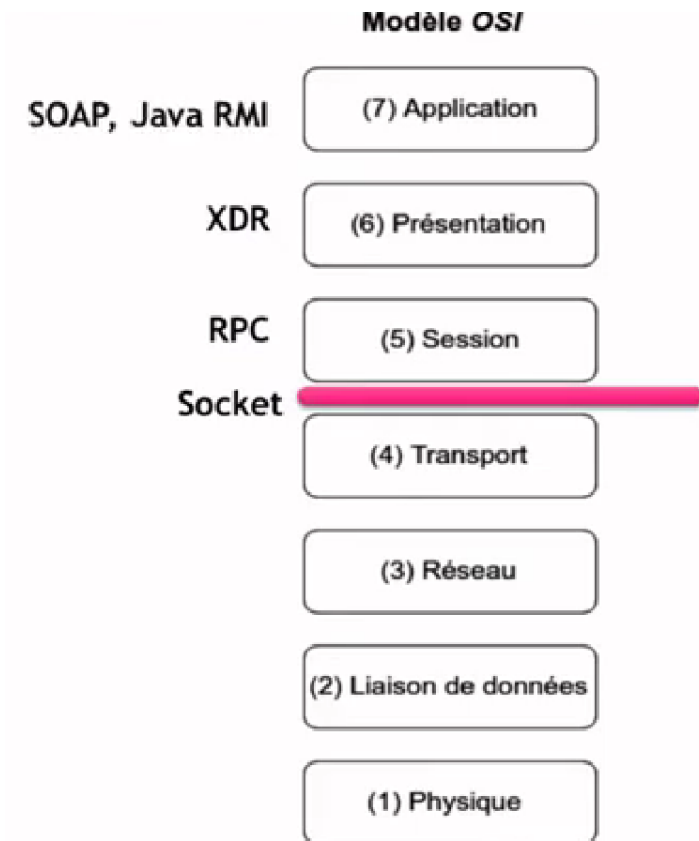
Ch. I. Introduction aux applications réparties

5. Outils de communication :

Comment faire une communication entre deux processus ?



- Outils de base : **les sockets (les prises, point de communication)**
- Les technologies d'appel de procédure à distances (**RPC**, remote Procedure call) regroupant les standards tels que CORBA, **RMI**, DCOM (Microsoft DNA), .NET, SOAP. Leur principe réside dans l'invocation d'un service (i.e. d'une procédure ou d'une méthode d'un objet) situé sur une machine distante indépendamment de sa localisation ou de son implémentation.



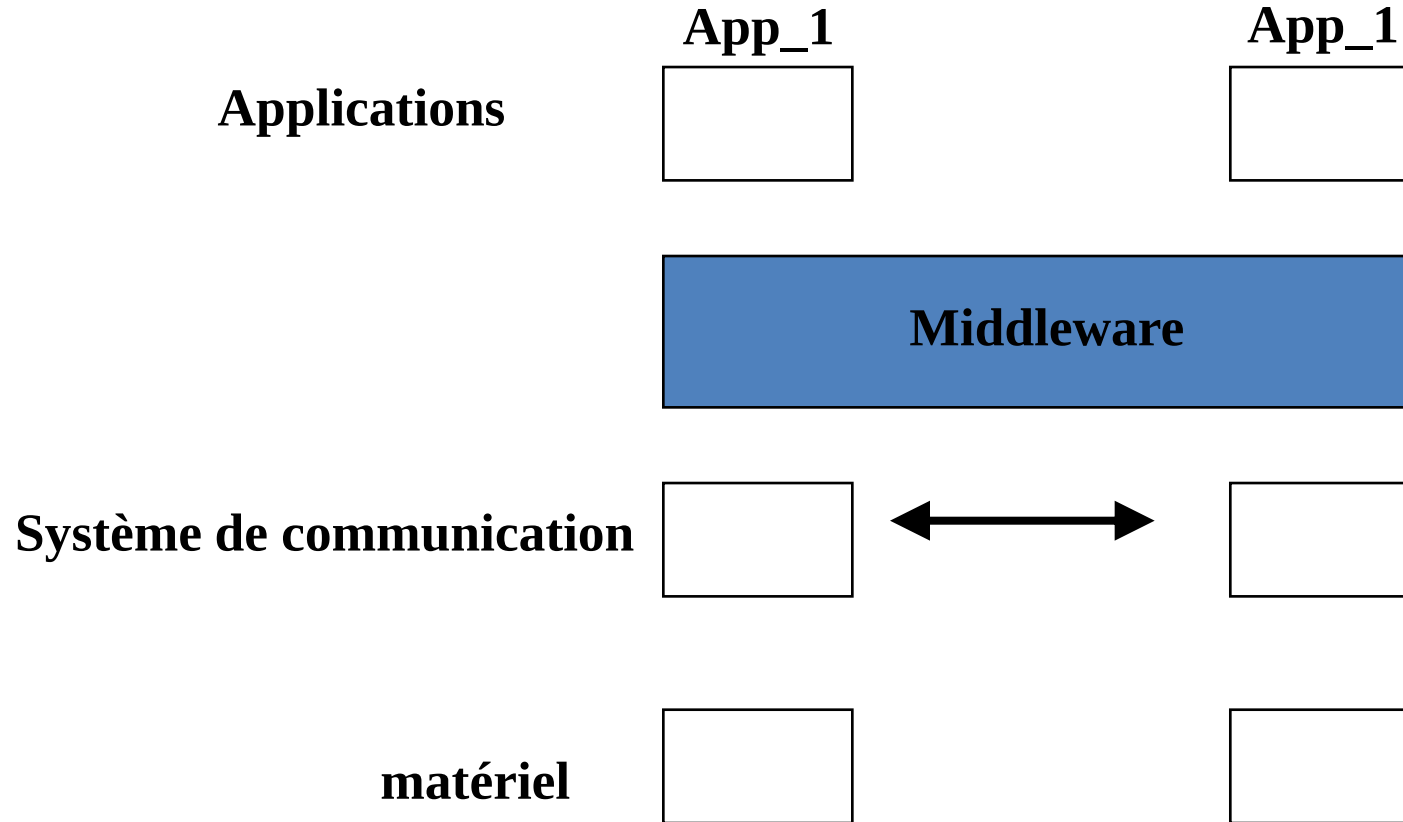
Ch. I. Introduction aux applications réparties

5. Outils de communication :

Un intergiciel (**middleware**) est un programme qui s'insère entre deux applications et qui va permettre de faire communiquer des machines entre elles, indépendamment de la nature du processeur, du système d'exploitation, du langage,Il est destiné à :

- Faciliter l'interopérabilité des applications : Unifier l'accès à des machines distantes
- Masquer l'hétérogénéité des machines & des systèmes : Être indépendant des systèmes d'exploitation et du langage de programmation des applications
- Faciliter la programmation réseau
- Masquer la répartition des traitements et des données
- Fournir des services associés pour gérer : Fiabilité, la Sécurité, la Scalabilité

Ch. I. Introduction aux applications réparties



Ch. I. Introduction aux applications réparties

Modes synchrones et asynchrones :

- **Mode synchrone** lorsque l'application cliente est bloquée en attente d'une réponse à une requête au serveur. Exemple: analogie du mode synchrone est le téléphone: les deux correspondants sont présents au moment de l'appel et chacun attend la réponse de l'autre pour parler.
 - ce mode permet de **s'assurer que le message a été envoyé et que l'on a bien reçu une réponse: on obtient plus de fiabilité**
- **Mode asynchrone** lorsque l'appel n'est pas bloquant : l'application **continue son déroulement sans attendre la réponse**. Exemple: semblable à la communication par courrier.
 - ce mode permet de libérer du temps de calcul pour d'autres actions afin de paralléliser les tâches: on obtient plus de performance.

Ch. I. Introduction aux applications réparties

Différents types des Middleware :

- Objets
 - JAVA RMI, CORBA (ORBIX, VisiBroker, OpenORB, ...)
 - DCOM
- Composant
 - J2EE (Websphere, Weblogic, JBOSS)
 - .Net
- Web-Service

Ch. I. Introduction aux applications réparties

Différents modèles d'exécution :

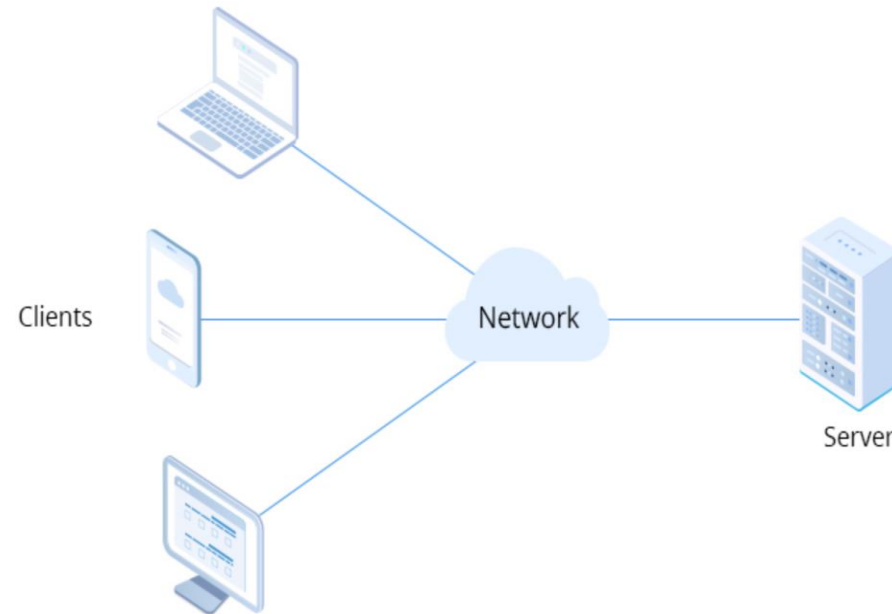
Le Modèle d'exécution client-serveur :

- Application client/serveur est application qui fait appel à **des services distants au travers d'un échange de messages** (les requêtes) plutôt que par **un partage de données** (mémoire ou fichiers).
- Dans un modèle client / serveur une application se divise en deux parties :
 - **serveur** : programme offrant un service sur un réseau (par extension, machine offrant un service). Donc c'est un programme qui s'exécute sur un réseau d'ordinateurs et qui est capable d'offrir un service. Le serveur reçoit une requête à travers le réseau, effectue le traitement nécessaire et retourne le résultat de la requête.
 - **client** : un programme qui demande de service (envoi la requête) au serveur et attend la réponse. Il est toujours l'initiateur du dialogue.

Ch. I. Introduction aux applications réparties

Différents modèles d'exécution :

Le Modèle d'exécution client-serveur :



Modèle client-serveur

Ch. I. Introduction aux applications réparties

Différents modèles d'exécution :

Le Modèle d'exécution client-serveur : (Avantages)

- **Gestion centralisée** : Toutes les données sont centralisées sur un serveur central, ce qui simplifie les contrôles en termes d'administration, de mise à jour des données et des logiciels.
- **Sécurité** : Les données et les informations sont stockées sur un seul serveur plutôt que sur plusieurs clients, permettant ainsi d'assurer une meilleure protection contre les menaces extérieures.
- **Évolutivité** : Il est possible d'ajouter ou supprimer des clients sans interrompre le fonctionnement normal des autres appareils.

Ch. I. Introduction aux applications réparties

Différents modèles d'exécution :

Le Modèle d'exécution client-serveur : (Inconvénients)

Le client-serveur a tout de même quelques inconvénients parmi lesquelles :

- **Coût élevé** : Les serveurs peuvent être coûteux à mettre en place et à maintenir.
- **En cas de panne** : une défaillance du serveur central risque de perturber tous les ordinateurs ou autres dispositifs du réseau client-serveur.
- **Congestion du trafic** : Lorsqu'un grand nombre de clients envoient des demandes au même serveur, celui-ci risque de ne pas être en mesure de supporter la charge, ce qui peut entraîner un problème de congestion du trafic.

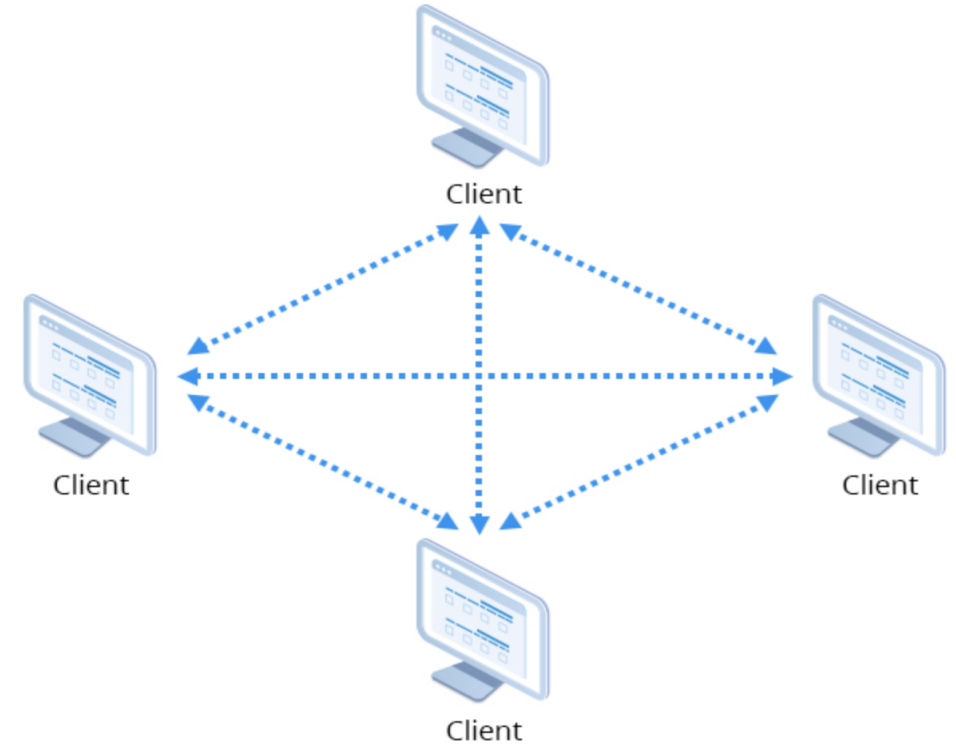
Ch. I. Introduction aux applications réparties

Différents modèles d'exécution :

Modèle peer to peer (pair à pair) :

Chaque ordinateur connecté au réseau est susceptible de jouer tour à tour le rôle de client et celui de serveur.

Peut être centralisée (les connexions passant par un serveur central intermédiaire) ou décentralisée (les connexions se faisant directement).



Modèle peer to pee

Ch. I. Introduction aux applications réparties

Différents modèles d'exécution :

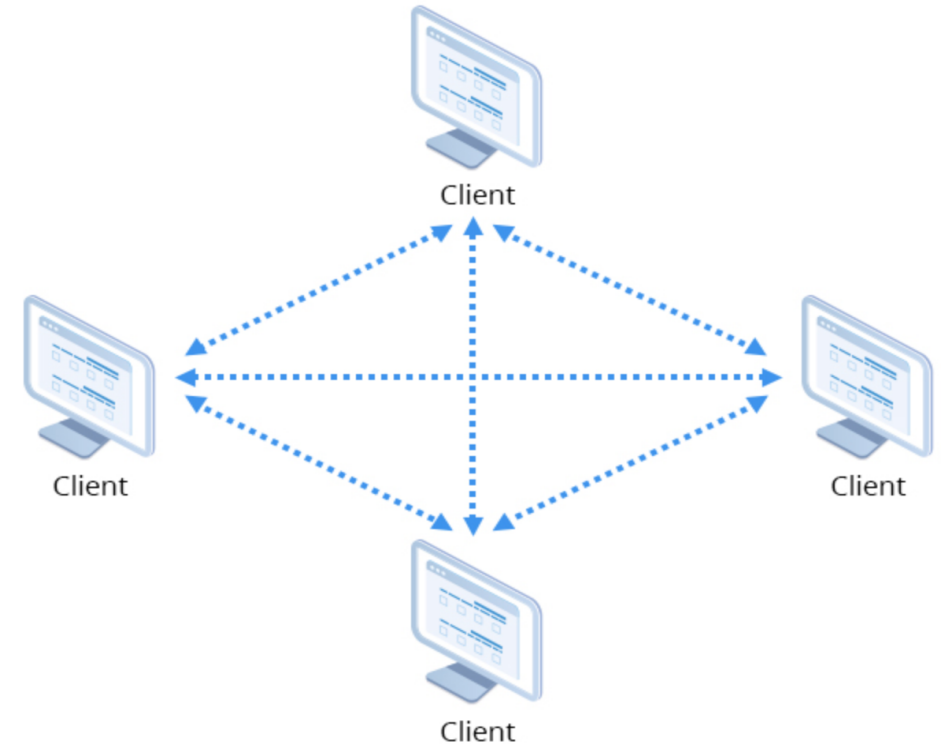
Modèle peer to peer (pair à pair) :

Avantage :

- Permet d'utiliser au mieux l'ensemble des ressources que représente chaque nœud d'un réseau.
- Permet un équilibre dans l'utilisation des ressources d'Internet tant au niveau de la bande-passante, que de l'espace de stockage ou de la capacité de traitement
- Réduire le coût du matériel utilisé

Inconvénients :

- La décentralisation des données et des services rend ce modèle très difficile à administrer
- La sécurité est moins facile à assurer, compte tenu des échanges transversaux ;



Modèle peer to pee

Ch. I. Introduction aux applications réparties

Résumé :

Les applications réparties sont des applications qui font communiquer plusieurs ordinateurs sans mémoire partagée.

On distingue les modes de communication RPC et par Message. Le mode synchrone est bloquant mais fiabilise les appels, à l'inverse du mode asynchrone, qui permet souvent des gains de performances.

Programmation avec les Sockets

Les Sockets TCP/UDP et leur mise
en oeuvre en C



02

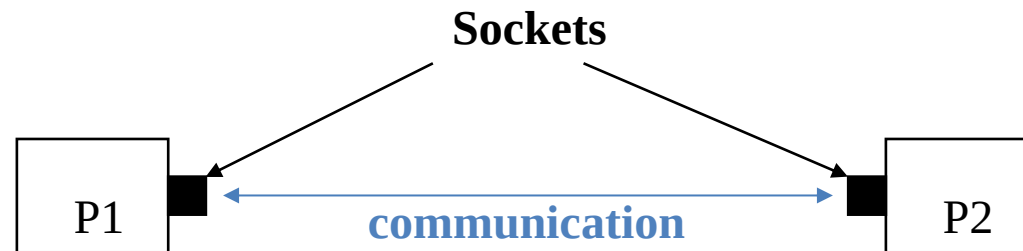
Ch. II. Programmation avec les Sockets

I. Les Sockets TCP/UDP et leur mise en œuvre en C :

1. Définition :

Un(e) socket désigne **un point de communication** par lequel un **processus peut envoyer ou recevoir** des informations.

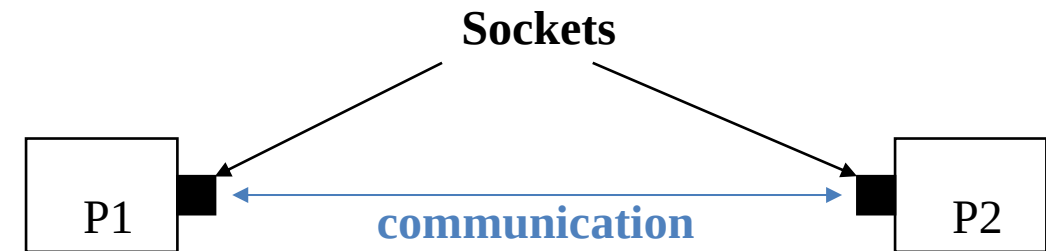
- Elle est associée à un processus et identifiée dans ce processus par un descripteur de socket «socket descriptor»
- Sous UNIX, ce point de communication est représenté par **une variable entière (int)**, **manipulée comme un descripteur de fichier**. Les sockets sont **l'outil de base (de bas niveau)** permettant la communication entre processus.



Ch. II. Programmation avec les Sockets

2. Principes de bases :

- Chaque processus **crée une socket**,
- Chaque socket sera associée à **un port** de sa machine hôte,
- Les deux sockets seront explicitement connectées **si on utilise un protocole en mode connecté ...**,
- Chaque processus lit et/ou écrit dans sa socket,
- Les données vont d'une socket à une autre à travers le réseau,
- Une fois terminé chaque machine **ferme sa socket**.



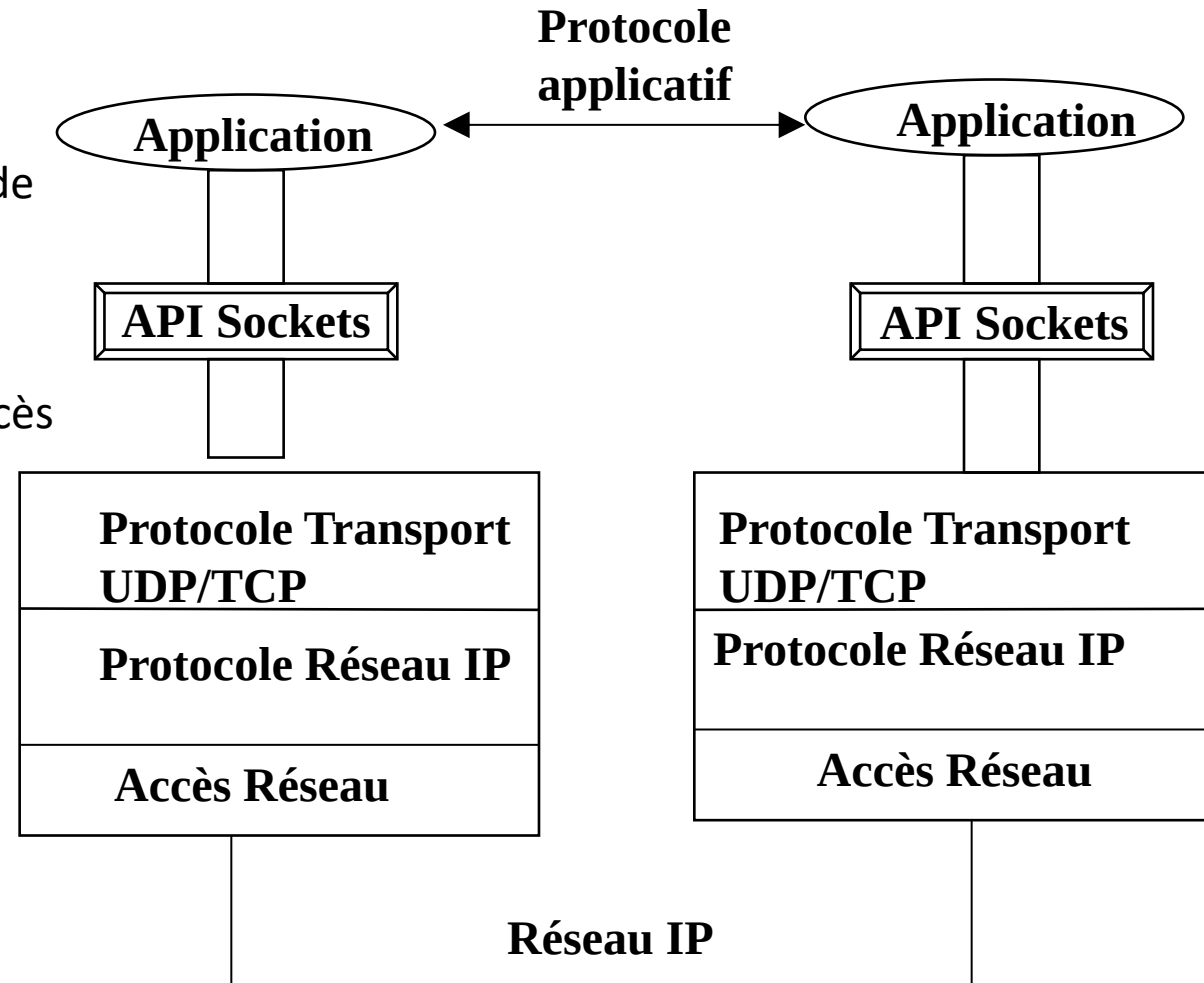
Ch. II. Programmation avec les Sockets

2. Principes de bases :

Le terme socket désigne aussi **une bibliothèque d'interface** de programmation (**pour les communications**) entre les applications et les couches réseau.

C'est un ensemble de primitives (fonctions/procédures) d'accès aux deux protocoles de transport d'Internet : **TCP et UDP**.

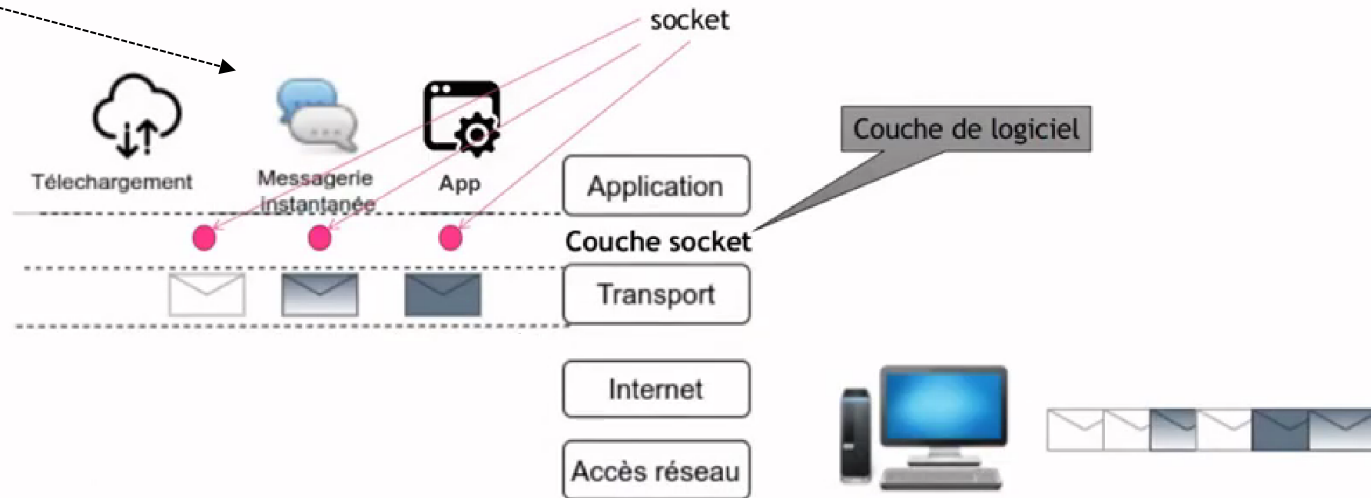
Il s'agit d'une interface souple mais de bas niveau.



Ch. II. Programmation avec les Sockets

2. Principes de bases :

Chaque application a sa propre Socket,



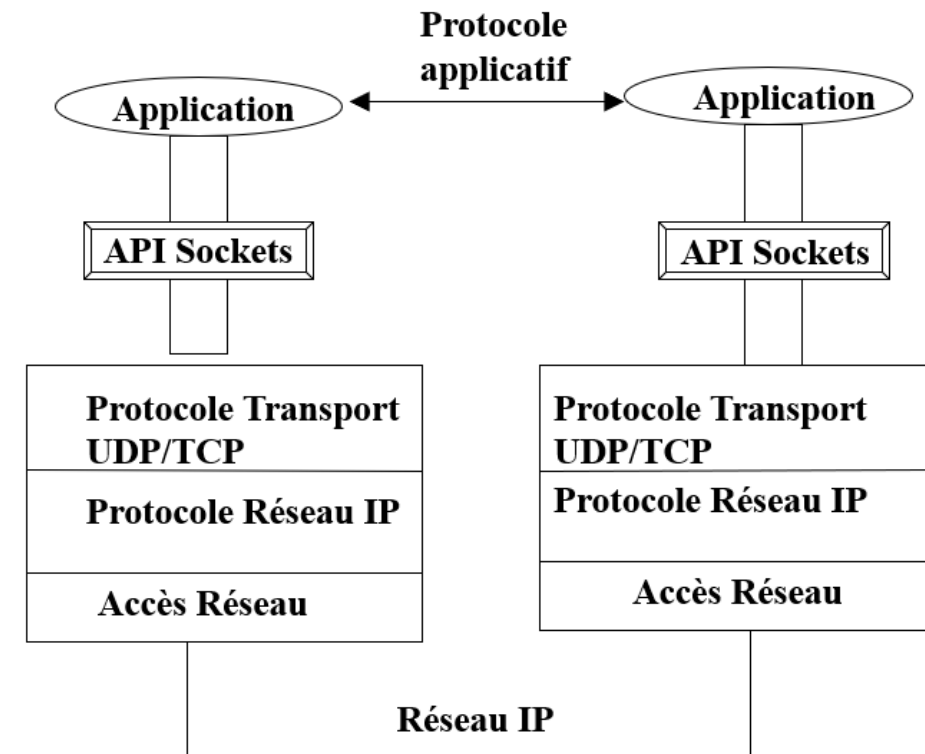
Ch. II. Programmation avec les Sockets

3. Expression des adresses des machines et des numéros de ports :

Les applications ne voient les couches de communication qu'à travers l'**API socket**.

Une communication met en jeu 2 « **extrémités** » identifiées par :

- **Le type de protocole (UDP ou TCP)**: La couche Transport utilise le numéro de port pour livrer les données au processus approprié sur l'ordinateur de destination
- **L'adresse IP**: La couche Internet utilise l'adresse IP pour transférer des données d'un ordinateur à un autre.
- **Le numéro de port** associé au processus
 - serveur : port local sur lequel les connexions sont attendues
 - client : allocation dynamique par le système



Ch. II. Programmation avec les Sockets

3. Expression des adresses des machines et des numéros de ports :

En résumé, une **socket** est un point d'accès aux couches réseau TCP/UDP.

Elle offre deux styles de communication:

- Streams (mode connecté → Protocole TCP)
- Datagrammes (mode non connecté → Protocole UDP)

Une **socket** est Liée localement à un port

Adressage de l'application sur le réseau: son couple **@IP : Port**

permet la **communication avec un port distant sur une machine distante** : c'est-à-dire avec une application distante

Ch. II. Programmation avec les Sockets

4. Les principaux protocoles de la couche transport :

Protocole UDP (User Datagram Protocol) :

- Fonctionnement en mode non connecté.
 - Pas besoin d'établir une connexion.
- **Transmission rapide des données mais n'est pas fiable**: les messages sont émis indépendamment les uns des autres.
 - ils peuvent prendre des routes différentes dans le réseau;
 - pas de garantie (les messages peuvent arriver dans le désordre, se perdre, ...).

Cas d'utilisation

- Cas où la perte d'une partie de ces données n'a pas grande importance (possibilité de substitution des données perdues)
 - Transmission de la voix sur IP/
 - Streaming (lecture en continue). Exemple: audio ou vidéo à la demande

Ch. II. Programmation avec les Sockets

4. Les principaux protocoles de la couche transport :

Protocole TCP (Transmission Control Protocol) :

- Fonctionnement en mode connecté.
 - Phase de connexion explicite avant la transmission de données (Établissement d'un tuyau virtuel entre deux processus avant transmissions).
- La transmission des données est fiable: Assure que
 - Les données envoyées sont toutes reçues par la machine destinataire.
 - Les données sont reçues dans l'ordre où elles ont été envoyées

Ch. II. Programmation avec les Sockets

5. Le concept de port :

Un port est une manière d'identifier différentes communications qui passent par une même interface réseau.

Il est:

- référencé par les processus pour établir des communications entre processus
- identifié sur une machine par un nombre entier (16 bits)
 - Les numéros de port p.n. (port numbers) sont locaux aux machines: un même p.n. existe sur différentes machines.

Ch. II. Programmation avec les Sockets

6. Le concept d'Adressage : port :

Adressage pour communication entre applications

- Adresse « réseau » d'une application noté : **@IP : port** ou **nomMachine:port**
- **Adresse IP** : identifiant de la machine sur laquelle tourne l'application
 - Exemple d'adresse IP: 192.129.12.34
- **Numéro de port** : identifiant local de l'application sur la Couche transport (TCP ou UDP). Ce numéro est un entier:
 - **de 0 à 1023** : ports réservés. Ils sont assignés par l'IANA (Internet Assigned Numbers Authority). Ils donnent accès aux services standard :
Exemple : 80 = HTTP, 21 = FTP, 53: DNS, ...
Exemple: 192.129.12.34:80 : accès au serveur Web tournant sur la machine d'adresse IP 192.129.12.34
 - **p.n. > 1024**
ports utilisateurs disponibles pour placer un service applicatif quelconque

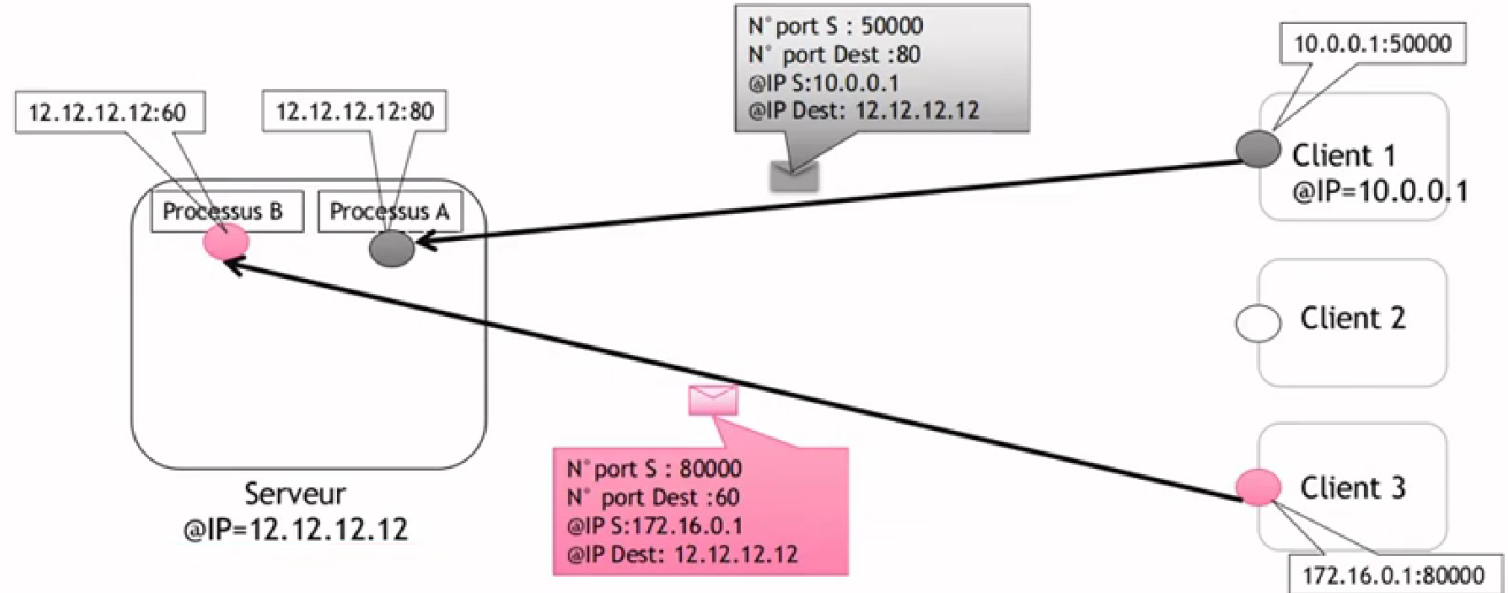
Ch. II. Programmation avec les Sockets

6. Le concept d'Adressage : port :

Socket pour décrire l'adresse d'un processus TCP ou UDP, C'est-à-dire une interface de connexion :

adresse IP + N°port

Socket = Canal de communication

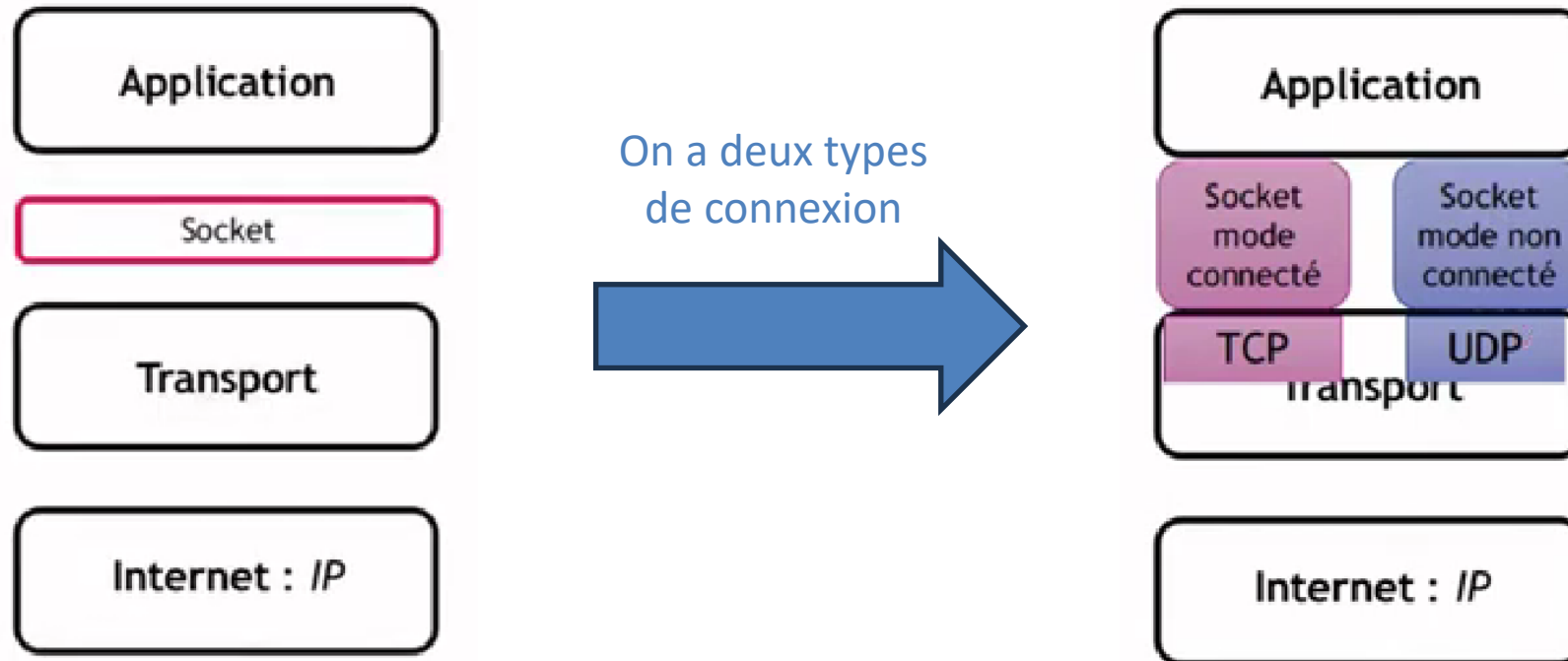


Exemple de deux sockets qui communique dans deux ports différents.

Ch. II. Programmation avec les Sockets

7. Les étapes de construction:

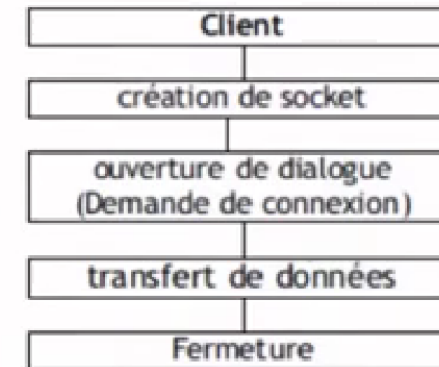
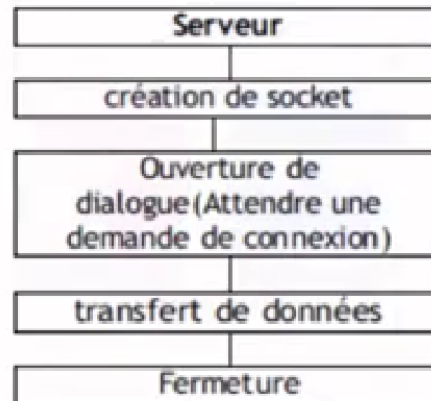
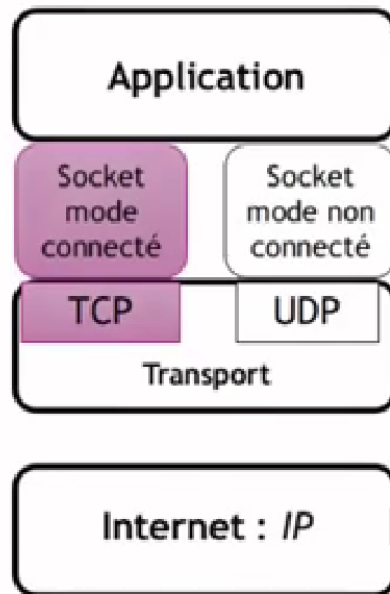
7.1. Création d'une socket :



Ch. II. Programmation avec les Sockets

7.1. Création d'une socket :

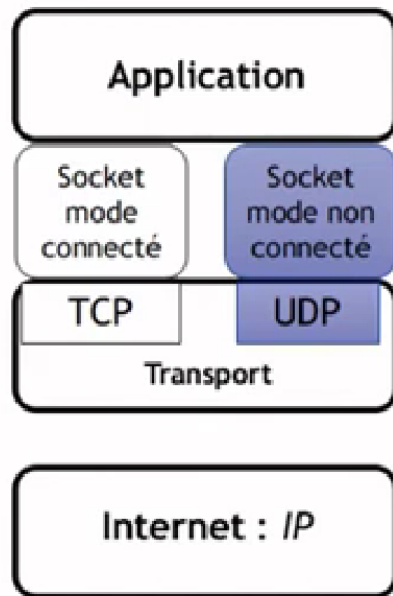
En mode connecte : (TCP)



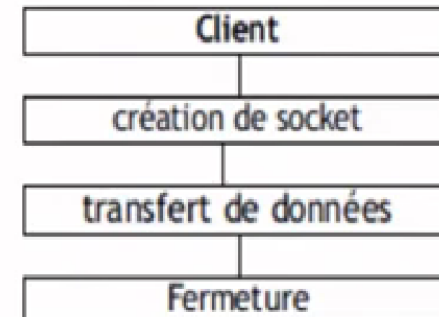
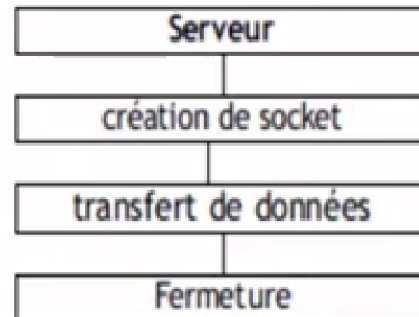
Ch. II. Programmation avec les Sockets

7.1. Création d'une socket :

En mode non connecté : (UDP)



Aucune liaison n'est établie. Les messages sont échangés individuellement.



Ch. II. Programmation avec les Sockets

7.1. Création d'une socket :

Identifiants d'une socket

- une adresse de socket est l'ensemble d'informations qui identifie une extrémité d'une association (une extrémité de l'application)
- les adresses de socket sont conservées dans des structures **sockaddr**. Le contenu de **sockaddr** dépend du protocole utilisé.
- Exemples de spécialisation
 - Contexte IP: communication distante
 - Fichier `<netinet/in.h>`
 - `struct sockaddr_in`
 - Contexte Unix : communication locale à la même machine via des sockets
 - Fichier `<sys/un.h>`
 - `struct sockaddr_un`

Ch. II. Programmation avec les Sockets

7.1. Création d'une socket :

Création d'une socket:

L'appel de la fonction **socket ()** crée un point de communication (une socket) pour permettre au processus de communiquer avec l'extérieur.

```
int socket (int domaine, int type, int protocol);
```

Ch. II. Programmation avec les Sockets

7.1. Création d'une socket :

- **domaine** : définit l'ensemble des sockets avec lesquelles la socket créée pourra communiquer (spécifie les entités avec lesquelles la communication sera réalisé) et le format des adresses possibles.

AF_INET : domaine **IP** (contexte **Internet**)

- Format des adresses: **sockaddr_in** (fichier standard <netinet/in.h>)

AF_UNIX: domaine **UNIX** (contexte **local**)

- Format des adresses: **sockaddr_un** (fichier standard <sys/un.h>)

Remarque: Les appels sockets pour le domaine AF_UNIX pour faire communiquer deux processus se trouvant sur une même machine sont identiques à ceux pour le domaine AF_INET, ce qui change c'est les structures associées aux adresses.

Ch. II. Programmation avec les Sockets

7.1. Création d'une socket :

- **type** : précise le protocole (le style) de communication:
 - **SOCK_DGRAM**: socket orienté vers la transmission de datagramme.
 - **SOCK_STREAM**: socket orienté vers l'échange de séquence continue de caractères (échange de flots d'octets).
 - **SOCK_RAW** : socket bas niveau (IP ou ICMP). Il permet l'accès à des protocoles de plus bas niveau, comme IP dans le domaine AF_INET, ou d'implanter de nouveaux protocoles.

Remarque: Si on utilise les sockets au-dessus d'UDP, la taille d'un échange est limitée à quelques kilo-octets: de deux Ko à huit Ko selon les systèmes.

Ch. II. Programmation avec les Sockets

7.1. Création d'une socket :

- **protocole** : spécifie le protocole à utiliser pour les communications, par exemple:
 - pour le type **SOCK_DGRAM** le seul protocole utilisé est **UDP** dans le domaine AF_INET
 - pour le type **SOCK_STREAM** le seul protocole utilisé est **TCP** dans le domaine AF_INET
- Dans le fichier <netinet/in.h> on définit les constantes symboliques pour identifier les deux protocoles UDP et TCP pour le domaine AF_INET.
 - La constante **IPPROTO.UDP** identifie le protocole **UDP**
 - La constante **IPPROTO.TCP** identifie le protocole **TCP**

Ch. II. Programmation avec les Sockets

7.1. Création d'une socket :

- Exemple :

```
#include <netinet/in.h>
s1=socket(AF_INET, SOCK_DGRAM, IPPROTO.UDP);
s2=socket(AF_INET, SOCK_STREAM, IPPROTO.TCP);
```

→ Dans la pratique, on remplace l'argument protocole **par la valeur 0** pour considérer le protocole par défaut associé à chaque type de socket.

Exemple :

```
// on utilise
        s1 =socket(AF_INET, SOCK_DGRAM, 0);
// au lieu de
        s1=socket(AF_INET, SOCK_DGRAM, IPPROTO.UDP);
```

Ch. II. Programmation avec les Sockets

7.1. Création d'une socket :

Valeurs de retour de la fonction `socket()` :

En cas de succès, l'appel de la fonction `socket()` retourne un **descripteur** sur la socket créée (un identificateur de socket). **Cet identificateur est local** à la machine et n'est pas connu par le destinataire.

En cas d'erreur, elle **retourne - 1**

Pour connaître le détail de l'erreur

On peut consulter la valeur de la variable `errno`. Liste des erreurs dans le fichier `<errno.h>`

On peut afficher un message avec la fonction `perror`

Ch. II. Programmation avec les Sockets

7.1. Création d'une socket :

Exemple de création d'un socket TCP mode connecte:

Cote Client :

```
int sd;  
if(sd = socket (AF_INET, SOCK_STREAM, 0)) < 0)  
{ printf("client : erreur de création de socket \n ");  
  exit(0);  
}
```

Cote Serveur :

```
int Ss;  
if(Ss = socket (AF_INET, SOCK_STREAM, 0)) < 0)  
{ printf("serveur : erreur de création de socket \n ");  
  exit(0);  
}
```

Ch. II. Programmation avec les Sockets

7.2. Nommage (attachement de la socket à une adresse) :

Après sa création,

- une socket est désignée par un descripteur obtenu au retour de l'appel de la fonction `socket()`.
- une socket n'est pas directement accessible par l'extérieur (en dehors du processus qui l'a créé).

Pour pouvoir être contactée de l'extérieur (accessible par l'extérieur) une socket doit être associée à une adresse. Le format de l'adresse est déterminé par le domaine de la socket

- Dans le domaine Unix :
 - une adresse = une entrée dans le système de fichiers
- Dans le domaine internet
 - une adresse = (adresse de la machine , numéro de port)

Ch. II. Programmation avec les Sockets

7.2. Nommage (attachement de la socket à une adresse) :

L'association `socket()` – `adresse` est réalisée par l'appel de la fonction `bind()`.

```
int bind ( int sock_desc, struct sockaddr *adresse, socklen_t longueur_adresse)
```

1. `sock_desc` = descripteur de la socket,
2. `*adresse` = adresse sur laquelle la socket sera liée. Pointeur sur la structure contenant les informations sur la machine locale (domaine, n° de port, et adresse IP)
3. `longueur_adresse` = taille de l'adresse. C'est la taille de la structure `sockaddr` obtenue avec la fonction `sizeof()`

Ch. II. Programmation avec les Sockets

7.2. Nommage (attachement de la socket à une adresse) :

La fonction retourne

- 0 en cas de succès (l'attachement a pu se réaliser)
- -1 en cas de problème

Problèmes les plus courants

- Adresse déjà utilisée (erreur EADDRINUSE)
- Liaison non autorisée (ex : port < 1024) sur ce port (erreur EACCES)

Ch. II. Programmation avec les Sockets

7.2. Nommage (attachement de la socket à une adresse) :

Déclaration typique d'une adresse :

- **struct sockaddr** est un identifiant « général » **qu'on ne l'utilise jamais directement** mais une spécialisation selon le type de réseau ou de communication utilisé.

Cas du domaine AF_UNIX: Le nom de la structure utilisée est **sockaddr_un** définie dans: /usr/include/sys/un.h

```
#include <sys/un.h>
struct sockaddr_un {
    short sun_family; /* AF_UNIX*/
    char sun_path[108] /* reference */
}
```

L'attachement d'une socket à une adresse ne peut se faire que si la référence correspondante n'existe pas.

Ch. II. Programmation avec les Sockets

7.2. Nommage (attachement de la socket à une adresse) :

Cas du domaine AF_INET : Le nom de la structure utilisée est **sockaddr_in** définie dans `/usr/include/netinet/in.h`

```
#include <netinet/in.h>
/* Adresse internet d'une socket */
struct sockaddr_in {
    short sin_family; /* AF_INET */
    u_short sin_port; /* numéro de port associé à la socket */
    struct in_addr sin_addr; /* adresse internet de la machine sur 32 bits */
    char sin_zero[8]; /* champs de 8 caractères nuls : inutilise */
};

/* Adresse Internet d'une machine */
struct in_addr {
    u_long s_addr;
}
```

Ch. II. Programmation avec les Sockets

7.2. Nommage (attachement de la socket à une adresse) :

Cas du domaine **AF_INET** :

Peut être aussi définie:

```
#include <netinet/in.h>
struct sockaddr_in{
    short sin_family;
    u_short sin_port;
    struct in_addr {
        u_long s_addr;
    } sin_addr;
    char sin_zero[8];
};
```


Ch. II. Programmation avec les Sockets

7.2. Nommage (attachement de la socket à une adresse) :

Les valeurs des champs `sin_addr` et `sin_port` peuvent avoir des valeurs particulières:

- `sin_family = AF_INET /* Domaine Internet */`
- `sin_addr.s_addr = INADDR_ANY`
- **INADDR_ANY** : permet d'utiliser n'importe quelle IP de la machine si elle en a plusieurs et/ou évite de connaître l'adresse IP locale.
- **sin_port = 0**, l'attachement peut se faire sur un port de numéro quelconque. Ceci est particulièrement intéressant pour les clients.

Remarque:

Les valeurs des champs **sin_addr**, **sin_port** et **s_addr** sont sous format réseau.

Ch. II. Programmation avec les Sockets

7.3. Représentations des nombres :

- Différences de représentation des nombres en mémoire
- Selon le système/matériel, **le codage binaire des nombres peut changer**

=> Pour éviter une mauvaise interprétation des nombres

On les code en « mode réseau » lorsqu'on les utilise dans les structures ou fonctions d'accès au réseau et sockets

Représentation des flottants (fixé par IEEE 754)

Ch. II. Programmation avec les Sockets

7.3. Représentations des nombres :

- Des fonctions de conversion définies dans **<netinet/in.h>** sont fournies pour permettre des conversions de représentations locales vers les représentations standard (les fonctions `htons` et `htonl`) et les conversions inverses (`ntohs` et `ntohl`).

```
#include <netinet/in.h>
u_short htons(u_short);
    // host-to-network, short integer (entier court local vers réseau)
u_long htonl(u_long);
    // host-to-network, long integer (entier long local vers réseau)
u_short ntohs(u_short);
    // network-to-host, short integer (entier court réseau vers local)
u_long ntohl(u_long);
    // network-to-host, long integer (entier long réseau vers local)
```

Ch. II. Programmation avec les Sockets

7.4. Identifiants d'une machine :

Le fichier `<netdb.h>`

```
struct hostent {  
    char *h_name;           // nom officiel,  
    char **h_aliases;       // liste des alias,  
    int h_addrtype;         // type d'adresse,  
    int h_length;           // longueur de l'adresse,  
    char **h_addr_list;     // liste des adresses  
    #define h_addr h_addr_list[0] // première adresse  
};
```

- Type d'adresse : internet (IP v4) par défaut
 - domaine = AF_INET, longueur = 4 (en octets)
- Une machine peut avoir plusieurs adresses IP et plusieurs noms

Ch. II. Programmation avec les Sockets

7.5. Accès aux identifiants de machines distantes :

La fonction **gethostbyname()** retourne: l'**identifiant de la machine** dont le nom est passé en paramètre.
NULL si aucune machine de ce nom est trouvée.

```
#include <netdb.h>  
struct hostent *gethostbyname(char *nom);
```

La structure retournée est placée à une adresse statique en mémoire.
Un nouvel appel de `gethostbyname()` écrase la valeur à cette adresse.
Si on veut la conserver, il est nécessaire de copier la structure avec un `memcpy()` ou un `bcopy()`.

Ch. II. Programmation avec les Sockets

7.5. Accès aux identifiants de machines distantes :

Exemple: Récupération du client de l'identifiant de la machine distante de nom "PGR"

```
static struct sockaddr_in addr_serveur; // identification de la socket d'écoute du serveur
struct hostent *host_serveur;           // identifiants de la machine où tourne le serveur
host_serveur = gethostbyname("PGR");    // récupération identifiants de la machine serveur

// création de l'identifiant de la socket d'écoute du serveur
bzero((char *) &addr_serveur, sizeof(addr_serveur));
//    #include <string.h>
//    void bzero (void *s, size_t n); // met à 0 les n premiers octets du bloc pointé par s.

addr_serveur.sin_family = AF_INET;
addr_serveur.sin_port = htons(4000); // le même numéro de port définie lors de l'appel de bind dans le serveur
memcpy(&addr_serveur.sin_addr.s_addr, host_serveur->h_addr, host_serveur->h_length);
```

Ch. II. Programmation avec les Sockets

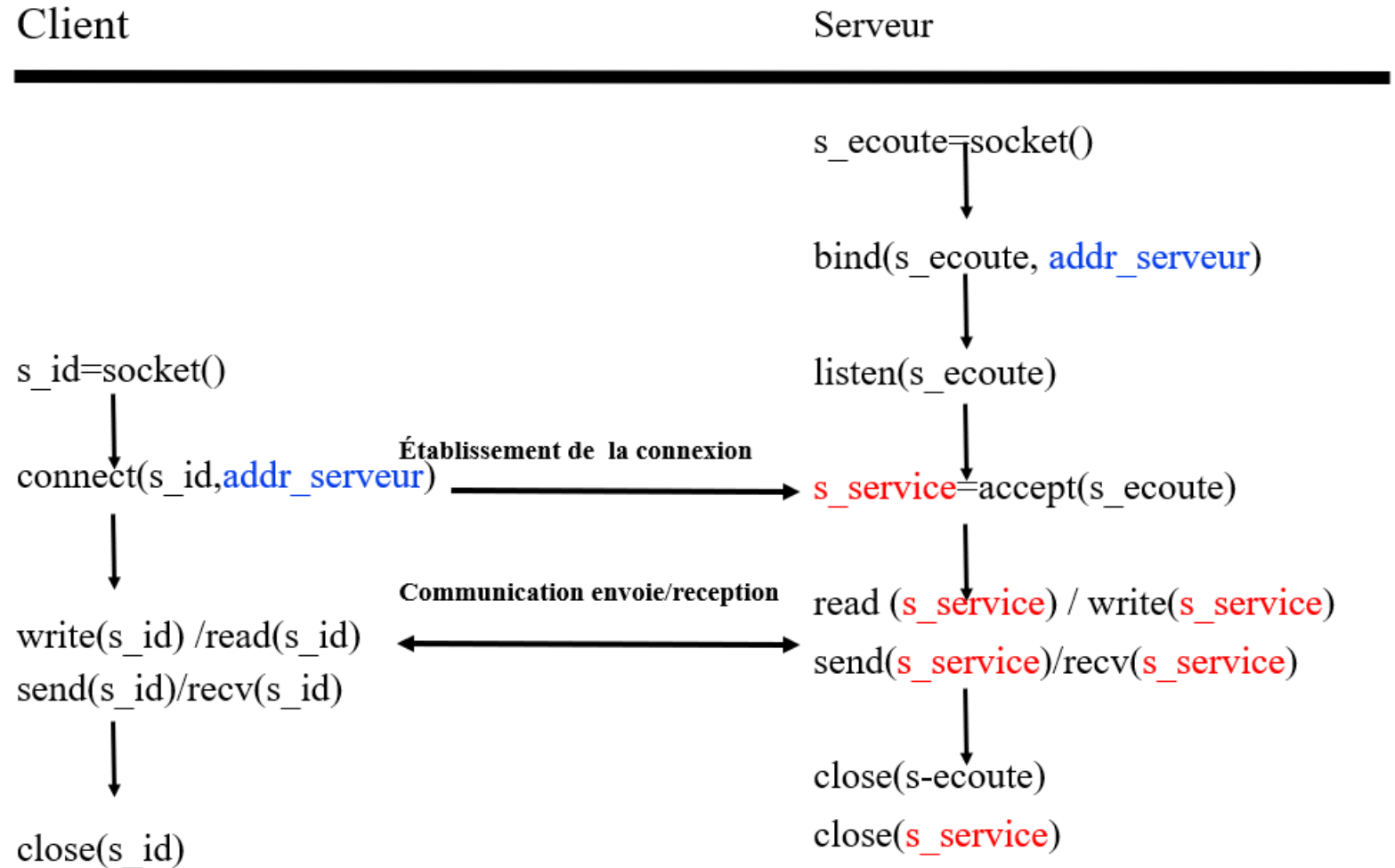
8 Communications en mode connecté (mode TCP):

1. Déroulement de la communication

- **L'appelé (ou serveur) :**
 - crée une socket (socket d'écoute);
 - associe une adresse socket (adresse Internet et numéro de port) au service: "binding";
 - se met en attente des connexions entrantes ;
 - pour chaque connexion entrante:
 - "accepte" la connexion (une nouvelle socket (socket de service) est créée);
 - lit ou écrit sur la nouvelle socket ;
 - ferme la nouvelle socket.
- **L'appelant (ou client) :**
 - crée une socket ;
 - se connecte au serveur en donnant l'adresse Internet du serveur et le numéro de port du service. Cette connexion attribue automatiquement un numéro de port au client;
 - lit ou écrit sur la socket ;
 - ferme la socket.

Ch. II. Programmation avec les Sockets

Communications en mode connecté (mode TCP):



Ch. II. Programmation avec les Sockets

8.1 Communications en mode connecté (mode TCP):

Remarque:

accept() retourne un nouveau descripteur de socket (socket de service), qui sera utilisé pour l'échange de données avec le client.

Sur quel type d'événement se bloquent ces fonctions:

- **Côté client :**
 - connect() attend qu'un serveur effectue un accept();
 - write() se bloque si le tampon d'émission est plein;
 - read() attend qu'un caractère au moins ait été reçu, à la suite d'une opération d'écriture effectuée par le serveur.
- **Côté serveur:**
 - accept() attend qu'un client effectue un connect();
 - read() attend un caractère, émis par un client;
 - write() se bloque si le tampon d'émission est plein.

Ch. II. Programmation avec les Sockets

8.2 Mise en place de canal de communication:

Coté client:

8.2.1.- Ouverture d'une connexion

L'appel de la fonction `connect()` permet d'ouvrir une connexion avec la partie serveur (connexion sur la socket d'écoute du serveur)

```
int connect(int s_id, struct sockaddr *addr_s_ecoute, int length_addr);
```

Paramètres

- `s_id` : descripteur de la socket coté client
- `addr_s_ecoute` : identifiant de la socket d'écoute coté serveur (socket_serveur)
- `length_addr` : taille de l'adresse utilisée

Retourne

- 0 en cas de succès,
- -1 sinon

Ch. II. Programmation avec les Sockets

8.2 Mise en place de canal de communication:

Coté serveur:

8.2.2.- Définition de la taille de la file d'attente sur une socket

La fonction `listen()` permet de configurer le nombre maximum de connexions en attente (le nombre maximum de connexions pendantes) à un instant donné.

```
int listen(int s_id, int nb_con_attente);
```

Paramètres

- **s_id** : descripteur de la socket d'écoute à configurer (socket chez le serveur)
- **nb_con_attente** : nombre max de connexions pendantes autorisées (taille de la file d'attente sur la socket chez le serveur). Le nombre de connexion pendantes doit être inférieur à la valeur de la constante SOMAXCONN (fichier <sys/socket.h>). Cette valeur dépend des systèmes d'exploitation

Exemple: SOMAXCONN = 128 sous Linux

Retourne

- 0 en cas de succès
- -1 en cas de problème

Ch. II. Programmation avec les Sockets

8.2 Mise en place de canal de communication:

Coté serveur:

8.2.3.- Attente d'une demande de connexion

accept() est la fonction qui attend la connexion d'un client sur une socket d'écoute

```
s_service= accept(int s_ecoute, struct sockaddr *addr_client, int *length_addr);
```

Paramètres

- **s_service** : c'est le numéro de la socket (appelée socket de service) renvoyé par l'appel de la fonction `accept()` dès qu'il reçoit une demande de connexion.
- **s_ecoute** : la socket d'écoute (à lier sur un port précis).
- **addr_client** : contiendra l'adresse du client qui se connectera
- **length_addr** : contiendra la taille de la structure d'address

Retourne un descripteur de socket

- La socket de service qui permet de communiquer avec le client qui vient de se connecter
- Retourne -1 en cas d'erreur
- Fonction bloquante jusqu'à l'arrivée d'une demande de connexion

Ch. II. Programmation avec les Sockets

8.3 Socket TCP : envoi/réception données:

- Si la connexion a réussi
 - un « **tuyau** » de communication directe est établi **entre la socket du coté client et la socket de service du coté serveur**
- Fonctions de communication
 - On peut utiliser les fonctions standards pour communiquer
 - **write()/send()** pour l'envoi
 - **read()/recv()** pour la reception

Ch. II. Programmation avec les Sockets

8.3 Socket TCP : envoi/réception données:

8.3.1. Envoie des données :

```
ssize_t write(int s_id, void *ptr_mem, size_t taille);  
ssize_t send(int s_id, void *ptr_mem, size_t taille, int option);
```

Paramètres

- **s_id** : descripteur de la socket
- **ptr_mem** : adresse de la zone mémoire des données à envoyer.
- **taille** : nombre d'octets à envoyer
- option prend la valeur 0

En cas de succès, elle retourne le nombre d'octets envoyés ou -1 en cas de problème

Ch. II. Programmation avec les Sockets

8.3 Socket TCP : envoi/réception données:

8.3.2. Réception des données :

```
ssize_t read (int s_id, void *ptr_mem, size_t taille);  
ssize_t recv (int s_id, void *ptr_mem, size_t taille, int option);
```

Paramètres

- **s_id** : descripteur de la socket
- **ptr_mem** : adresse du début de la zone mémoire où seront écrites les données reçues.
- **taille** : taille de la zone mémoire
- option prend la valeur 0

Retourne le nombre d'octets reçus ou -1 en cas de pb

Ch. II. Programmation avec les Sockets

8.3 Socket TCP : envoi/réception données:

8.3.3. Fermeture de la socket:

- La fonction **close()** gère la fermeture de la connexion au niveau système (fichiers).
 int **close**(int **s_id**)
 s_id : descripteur de socket
- Le noyau essaie d'envoyer les données non encore émises avant de sortir du **close()**.

Ch. II. Programmation avec les Sockets

8.4 Exemple:

- Serveur attend la connexion du client
- Client
 - envoie la chaîne **“Bonjour c'est le Client”** au serveur
 - et ensuite attend une réponse
- Serveur
 - reçoit la chaîne **“Bonjour c'est le Client”**

Travail en local (**Mode AF_UNIX**)

on écrit 2 programmes qui communiquent `socket_clientTCP_Unix.c` et `socket_serveurTCP_Unix.c`

Ch. II. Programmation avec les Sockets

Coté serveur: fichier socket_serveurTCP_UNIX.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>
#define SOCKET_PATH "/tmp/unix_socket"
#define BUFFER_SIZE 256

int main() {
    int server_fd, client_fd;
    struct sockaddr_un server_addr, client_addr;
    socklen_t client_len;
    char buffer[BUFFER_SIZE];
    // Création du socket
    if ((server_fd = socket(AF_UNIX, SOCK_STREAM, 0)) == -1) {
        perror("Socket error");    exit(EXIT_FAILURE);
    }
```

Ch. II. Programmation avec les Sockets

Coté serveur: fichier socket_serveurTCP_UNIX.c

```
// Configuration de l'adresse du socket
memset(&server_addr, 0, sizeof(server_addr));
server_addr.sun_family = AF_UNIX;
strncpy(server_addr.sun_path, SOCKET_PATH, sizeof(server_addr.sun_path) - 1);

// Liaison
unlink(SOCKET_PATH);
if (bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) == -1) {
    perror("Bind error");
    close(server_fd);
    exit(EXIT_FAILURE);
}
```

Ch. II. Programmation avec les Sockets

Coté serveur: fichier socket_serveurTCP_UNIX.c

```
// Écoute des connexions
if (listen(server_fd, 5) == -1) {
    perror("Listen error");
    close(server_fd);
    exit(EXIT_FAILURE);
}
printf("Serveur AF_UNIX en attente de connexion...\n");

// Acceptation des connexions
client_len = sizeof(client_addr);
if ((client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len)) == -1) {
    perror("Accept error");
    close(server_fd);
    exit(EXIT_FAILURE);
}
```

Ch. II. Programmation avec les Sockets

Coté serveur: fichier socket_serveurTCP_UNIX.c

```
// Réception des données
memset(buffer, 0, BUFFER_SIZE);
if (read(client_fd, buffer, BUFFER_SIZE) > 0) {
    printf("Recu: %s\n", buffer);
}

// Fermeture
close(client_fd);
close(server_fd);
unlink(SOCKET_PATH);
return 0;
}
```

Ch. II. Programmation avec les Sockets

Coté client: fichier socket_clientTCP_UNIX.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>
#define SOCKET_PATH "/tmp/unix_socket"
#define BUFFER_SIZE 256

int main() {
    int client_fd;
    struct sockaddr_un server_addr;
    char buffer[BUFFER_SIZE] = "Bonjour c'est le Client";
    // Création du socket
    if ((client_fd = socket(AF_UNIX, SOCK_STREAM, 0)) == -1) {
        perror("Socket error");
        exit(EXIT_FAILURE);
    }
```


Ch. II. Programmation avec les Sockets

Coté client: fichier socket_clientTCP_UNIX.c

```
// Configuration de l'adresse du socket
memset(&server_addr, 0, sizeof(server_addr));
server_addr.sun_family = AF_UNIX;
strncpy(server_addr.sun_path, SOCKET_PATH, sizeof(server_addr.sun_path) - 1);

// Connexion au serveur
if (connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) == -1) {
    perror("Connect error");
    close(client_fd);
    exit(EXIT_FAILURE);
}
```

Ch. II. Programmation avec les Sockets

Coté client: fichier socket_clientTCP_UNIX.c

```
// Envoi des données
if (write(client_fd, buffer, strlen(buffer)) == -1) {
    perror("Write error");
    close(client_fd);
    exit(EXIT_FAILURE);
}

printf("Message envoye au serveur...\n");

// Fermeture
close(client_fd);
return 0;
}
```




THANK YOU